Progetto: linee guida e mia idea

- Linee guida generali
  - Project 1: Kafka
- Mia idea: tracciatore real-time di partenze di voli (nome da definire)
  - Esempio informazioni con API Schiphol
  - Struttura directory lavoro e spiegazione
    - File a livello radice
    - Cartella `security/`
    - Cartella `scripts/`
    - Cartella `services/`
    - `services/API-producer/Dockerfile`
    - `services/API-producer/requirements.txt`
    - `services/storage/storage.py`
    - `services/storage/Dockerfile` + `requirements.txt`
    - `services/notifier/notifier.py`
    - `services/notifier/Dockerfile` + `requirements.txt`
    - `services/stats-aggregator/aggregator.py`
    - `services/stats-aggregator/Dockerfile` + `requirements.txt`
    - `services/dashboard/backend.py`
    - `services/dashboard/template/index.html`
    - `services/dashboard/static/script.js`
    - `services/dashboard/static/style.css`
    - `services/dashboard/Dockerfile` + `requirements.txt`
  - Ordine di sviluppo consigliato
    - Fase 1 — Fondazioni infrastrutturali
    - Fase 2 — Ingestion
    - Fase 3 — Primo consumer (il più semplice)
    - Fase 4 — Il poller vero
    - Fase 5 — Servizi dual-role
    - Fase 6 — Dashboard
    - Fase 7 — Demo dei requisiti non funzionali
    - Fase 8 — Estensioni future

Linee guida generali

The exam consists of a project and an oral test. The project aims to deepen the practical aspects of one or more topics discussed during the course. In this page you can find several project proposals that can be categorized in two classes:

1. **Practical project**: the practical application of some of the concepts learned during classes. It might require coding, development, system configuration, system deployment. A written report of max 5 pages must describe the implemented project. A presentation has to be done during the oral exam, presenting all implementation choices and details. Demo of the project in operation.

A written report describing activities in the project must be sent to the professor 2 weeks before the examination day (more details in the following posts). The evaluation of the project, expressed with a pass/fail mark, considers its quality, soundness, and the level of mastery of the topics, as well as the clarity of the report. The oral test, which is accessed after a positive evaluation of the project, is based on the discussion of some topics covered in the course and on in-depth questions about the presented project. The evaluation of the oral test, expressed on a scale between 0 and 30, takes into account the level of mastery of the topics, clarity, and language skills. Students should refer to the Web pages of the remaining modules for the description of the corresponding exam modalities.

The oral exam aims to verify the knowledge of the student about all arguments discussed during the course, with particular reference to those concepts analyzed in the project. The final evaluation, expressed in thirtieths, is computed considering: the knowledge of the topics, ability of applying the learned knowledge to the resolution of a practical project, project quality, critical thinking skills, clarity of exposition, and property of language.

Project 1: Kafka

- Install and configure Kafka so that some of the important non-functional properties of a distributed system are preserved (e.g., fault tolerance and HA).
- Enable advanced security features (e.g., channel encryption, authentication)
- Develop a simple microservice application using kafka queue to demonstrate the obtained
  functionalities (e.g., a microservice that writes data and one that reads data for a specific purpose for which a queue is mandatory instead of another messaging system).

- During the demonstration it is also needed to show the non-functional properties. For instance, to show robustness to failures (e.g., simulating faults or security attacks).

Mia idea: tracciatore real-time di partenze di voli (nome da definire)

L'obiettivo del mio progetto è sviluppare, tramite Kafka e Docker (linguaggio scelto: python) un tracciatore real-time di partenze di voli da 3 aeroporti diversi:

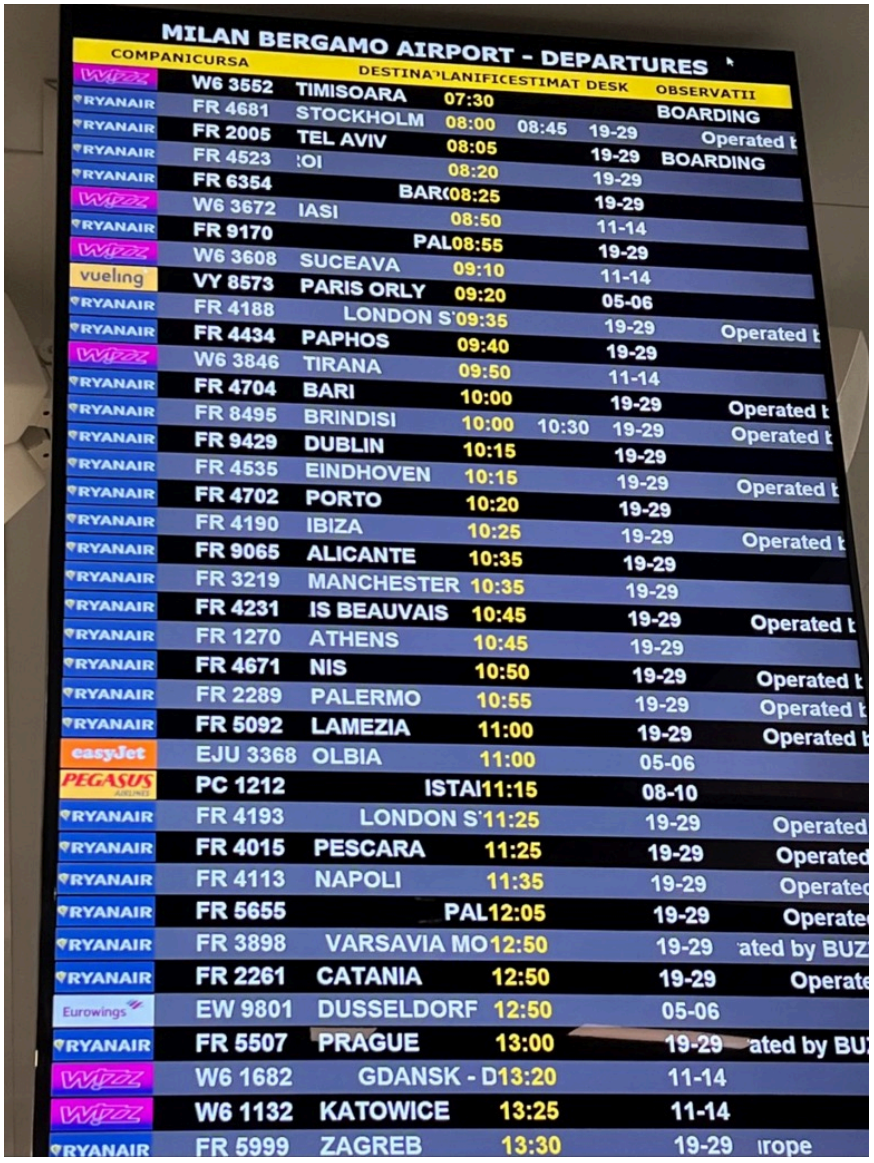
- Amsterdam Schiphol (AMS)
- da decidere
- da decidere

La scelta di questi 3 aeroporti è legata alla disponibilità, direttamente dagli stessi, di informazioni in tempo reale (o anche in futuro/passato) sui voli in partenza e arrivo. I dati che verranno mostrati interesseranno SOLAMENTE i regolari voli passeggeri (vanno quindi esclusi voli cargo e privati) e bisognerà evitare di far vedere voli in code sharing: in quest'ultimo caso, dovranno essere mostrati solamente i dati della compagnia che opera effettivamente il volo.

La dashboard principale sarà composta da 3 riquadri, uno per aeroporto, dove in cima viene mostrato:

- Nome dell'aeroporto
- Numero totale voli partiti da inizio giornata
- Numero totale voli arrivati da inizio giornata
- Ritardo medio giornaliero (basandosi sul giorno corrente dell'aeroporto) accumulato sulle partenze
- Ritardo medio giornaliero (basandosi sul giorno corrente dell'aeroporto) accumulato sugli arrivi
- Se possibile: condizioni meteo all'aeroporto (temperatura, sole/nuvoloso/pioggia/neve ecc, intensità vento)

Al di sotto di queste informazioni, viene mostrato un classico tabellone delle partenze (nel caso espando con anche gli arrivi, ma per ora inizierei con solo le partenze) come questo qui sotto:



I dati che andranno mostrati saranno:

- Nome compagnia (solo nome testuale, no logo come in questo tabellone)
- Codice volo
- Destinazione
- Gate partenza
- Orario partenza schedulato
- Orario partenza effettivo/stimato
- Stato volo (ad esempio BOARDING NOW, GATE CLOSED, ecc...) Questa voce dipenderà dalle API

Esempio informazioni con API Schiphol

Codice

```
#!/usr/bin/env python3
"""
Schiphol Public Flight API — test script.

Scarica le PARTENZE da Schiphol (AMS) di OGGI tra le 10:00
e le 14:00 (ora locale).

Prerequisiti:
  1. Registrarsi su https://developer.schiphol.nl/signup e
creare una Application
     per ottenere app_id e app_key.
  2. Esportare le credenziali come variabili d'ambiente:
        export SCHIPHOL_APP_ID="..."
        export SCHIPHOL_APP_KEY="..."
  3. pip install requests
  4. python schiphol_departures_test.py
"""

import json
import os
import sys
from datetime import date

import requests

API_URL = "https://api.schiphol.nl/public-flights/flights"
RESOURCE_VERSION = "v4"   # versione attualmente pubblica
(Public Flight V4 API)

# --- Credenziali ----------------------------------------
-------------------
APP_ID = "16a1d0e9"
APP_KEY = "6e909dec5307824655e44fa75007f938"
if not APP_ID or not APP_KEY:
    sys.exit("ERRORE: esporta le variabili SCHIPHOL_APP_ID
```

```
e SCHIPHOL_APP_KEY")

# --- Query ------------------------------------------------
-------------------
today = date.today().isoformat()   # es. "2026-04-22"

headers = {
    "app_id": APP_ID,
    "app_key": APP_KEY,
    "ResourceVersion": RESOURCE_VERSION,
    "Accept": "application/json",
}

params = {
    "flightDirection": "D",                      # D = Dep
artures, A = Arrivals
    "fromDateTime": f"{today}T10:00:00",
    "toDateTime":   f"{today}T14:00:00",
    "searchDateTimeField": "scheduleDateTime",    # filtra
sullo scheduled time
    "sort": "+scheduleTime",                     # ordina
dal più vicino al più lontano
    "includedelays": "true",
    "page": 0,
}

# --- Chiamata con paginazione (il server risponde con head
er Link: next) ----
all_flights = []
url = API_URL
first_request = True

while url:
    resp = requests.get(
        url,
        headers=headers,
        params=params if first_request else None,   # sulle
pagine successive non serve
```

```
        timeout=15,
    )
    first_request = False

    # Alcune info utili su quota/ratelimit, se il server le
espone
    for h in ("X-RateLimit-Limit", "X-RateLimit-Remaining",
"X-RateLimit-Reset"):
        if h in resp.headers:
            print(f"{h}: {resp.headers[h]}")

    if resp.status_code != 200:
        print(f"HTTP {resp.status_code} — {resp.text[:40
0]}")
        break

    page_flights = resp.json().get("flights", [])
    all_flights.extend(page_flights)
    print(f"Pagina scaricata: {len(page_flights)} voli (tot
ale finora: {len(all_flights)})")

    # requests parsa automaticamente l'header Link in un di
ct
    next_link = resp.links.get("next")
    url = next_link["url"] if next_link else None

print(f"\n=== TOTALE: {len(all_flights)} partenze tra le 1
0:00 e le 14:00 di oggi ===\n")

# --- Vista rapida dei campi principali ------------------
-------------------
for f in all_flights[:10]:
    dest   = ",".join(f.get("route", {}).get("destination
s", []))
    states = f.get("publicFlightState", {}).get("flightStat
es", [])
    print(
        f"{f.get('flightName','')}:8} | "
```

```
        f"sched {f.get('scheduleTime','-'):5} -> {dest:4} |
"
        f"gate={f.get('gate','-'):>4} | "
        f"term={f.get('terminal','-')!s:>4} | "
        f"state={states}"
    )

# --- Dump grezzo del primo volo per ispezione ------------
------------------
if all_flights:
    print("\n=== JSON GREZZO del primo volo ===")
    print(json.dumps(all_flights[0], indent=2, ensure_ascii
=False))Ri
```

Risultato

```
Pagina scaricata: 20 voli (totale finora: 20)
Pagina scaricata: 20 voli (totale finora: 40)
Pagina scaricata: 20 voli (totale finora: 60)
Pagina scaricata: 20 voli (totale finora: 80)
Pagina scaricata: 20 voli (totale finora: 100)
Pagina scaricata: 20 voli (totale finora: 120)
Pagina scaricata: 20 voli (totale finora: 140)
Pagina scaricata: 20 voli (totale finora: 160)
Pagina scaricata: 20 voli (totale finora: 180)
Pagina scaricata: 20 voli (totale finora: 200)
Pagina scaricata: 20 voli (totale finora: 220)
Pagina scaricata: 20 voli (totale finora: 240)
Pagina scaricata: 20 voli (totale finora: 260)
Pagina scaricata: 20 voli (totale finora: 280)
Pagina scaricata: 20 voli (totale finora: 300)
Pagina scaricata: 20 voli (totale finora: 320)
Pagina scaricata: 20 voli (totale finora: 340)
Pagina scaricata: 20 voli (totale finora: 360)
Pagina scaricata: 20 voli (totale finora: 380)
Pagina scaricata: 20 voli (totale finora: 400)
Pagina scaricata: 20 voli (totale finora: 420)
Pagina scaricata: 20 voli (totale finora: 440)
Pagina scaricata: 20 voli (totale finora: 460)
Pagina scaricata: 20 voli (totale finora: 480)
Pagina scaricata: 20 voli (totale finora: 500)
Pagina scaricata: 20 voli (totale finora: 520)
Pagina scaricata: 20 voli (totale finora: 540)
Pagina scaricata: 20 voli (totale finora: 560)
Pagina scaricata: 18 voli (totale finora: 578)

=== TOTALE: 578 partenze tra le 10:00 e le 14:00 di oggi ===
```

```
AC9074   | sched 10:00:00 -> FRA  | gate= B31 | term=   1 | state=['DEP']
AI8542   | sched 10:00:00 -> FRA  | gate= B31 | term=   1 | state=['DEP']
NH5411   | sched 10:00:00 -> FRA  | gate= B31 | term=   1 | state=['DEP']
UA9195   | sched 10:00:00 -> FRA  | gate= B31 | term=   1 | state=['DEP']
LH987    | sched 10:00:00 -> FRA  | gate= B31 | term=   1 | state=['DEP']
KL1775   | sched 10:00:00 -> BER  | gate= C12 | term=   1 | state=['DEP']
DL9190   | sched 10:00:00 -> BER  | gate= C12 | term=   1 | state=['DEP']
GA9252   | sched 10:00:00 -> BER  | gate= C12 | term=   1 | state=['DEP']
KQ1823   | sched 10:00:00 -> BER  | gate= C12 | term=   1 | state=['DEP']
MF9331   | sched 10:00:00 -> BER  | gate= C12 | term=   1 | state=['DEP']

=== JSON GREZZO del primo volo ===
{
  "lastUpdatedAt": "2026-04-22T10:05:56.485+02:00",
  "actualOffBlockTime": "2026-04-22T09:55:49.000+02:00",
  "aircraftRegistration": "DAIUN",
  "aircraftType": {
    "iataMain": "32S",
    "iataSub": "320"
  },
  "checkinAllocations": {
    "checkinAllocations": [
      {
        "endTime": "2026-04-22T09:15:00.000+02:00",
        "rows": {
          "rows": [
            {
              "position": "2",
              "desks": {
                "desks": [
                  {
                    "position": 1
                  },
                  {
                    "position": 2
                  },
                  {
                    "position": 3
                  },
                  {
                    "position": 4
                  },
                  {
                    "position": 5
                  },
                  {
                    "position": 6
                  },
                  {
                    "position": 7
                  },
                  {
                    "position": 8
                  },
                  {
                    "position": 9
```

```
                },
                {
                  "position": 10
                },
                {
                  "position": 11
                },
                {
                  "position": 12
                },
                {
                  "position": 13
                }
              ]
            }
          },
          {
            "position": "3",
            "desks": {
              "desks": [
                {
                  "position": 1
                },
                {
                  "position": 2
                },
                {
                  "position": 3
                },
                {
                  "position": 4
                },
                {
                  "position": 5
                },
                {
                  "position": 6
                },
                {
                  "position": 7
                },
                {
                  "position": 8
                },
                {
                  "position": 9
                },
                {
                  "position": 10
                }
              ]
            }
          },
          {
            "position": "4",
            "desks": {
```

```
                "desks": [
                  {
                    "position": 1
                  },
                  {
                    "position": 2
                  },
                  {
                    "position": 3
                  }
                ]
              }
            }
          ]
        },
        "startTime": "2026-04-22T07:30:00.000+02:00"
      },
      {
        "endTime": "2026-04-22T09:30:00.000+02:00",
        "rows": {
          "rows": [
            {
              "position": "2",
              "desks": {
                "desks": [
                  {
                    "position": 1
                  },
                  {
                    "position": 2
                  },
                  {
                    "position": 3
                  },
                  {
                    "position": 4
                  },
                  {
                    "position": 5
                  },
                  {
                    "position": 6
                  },
                  {
                    "position": 7
                  },
                  {
                    "position": 8
                  },
                  {
                    "position": 9
                  },
                  {
                    "position": 10
                  }
                ]
```

```
              }
            },
            {
              "position": "3",
              "desks": {
                "desks": [
                  {
                    "position": 1
                  },
                  {
                    "position": 2
                  },
                  {
                    "position": 3
                  },
                  {
                    "position": 4
                  },
                  {
                    "position": 5
                  },
                  {
                    "position": 6
                  },
                  {
                    "position": 7
                  },
                  {
                    "position": 8
                  },
                  {
                    "position": 9
                  },
                  {
                    "position": 10
                  }
                ]
              }
            }
          ]
        },
        "startTime": "2026-04-22T09:15:00.000+02:00"
      }
    ]
  },
  "codeshares": {
    "codeshares": [
      "AC9074",
      "AI8542",
      "NH5411",
      "UA9195"
    ]
  },
  "expectedTimeBoarding": "2026-04-22T09:38:00.000+02:00",
  "expectedTimeGateClosing": "2026-04-22T09:55:00.000+02:00",
  "expectedTimeGateOpen": "2026-04-22T09:20:00.000+02:00",
```

```
  "expectedSecurityFilter": "VF1",
  "flightDirection": "D",
  "flightName": "AC9074",
  "flightNumber": 9074,
  "gate": "B31",
  "pier": "B",
  "id": "144763328367556495",
  "isOperationalFlight": true,
  "mainFlight": "LH987",
  "prefixIATA": "AC",
  "prefixICAO": "ACA",
  "airlineCode": 54,
  "publicFlightState": {
    "flightStates": [
      "DEP"
    ]
  },
  "route": {
    "destinations": [
      "FRA"
    ],
    "eu": "S",
    "visa": false
  },
  "scheduleDateTime": "2026-04-22T10:00:00.000+02:00",
  "scheduleDate": "2026-04-22",
  "scheduleTime": "10:00:00",
  "serviceType": "J",
  "terminal": 1,
  "transferPositions": {
    "transferPositions": [
      3
    ]
  },
  "schemaVersion": "4"
}
```


Struttura directory lavoro e spiegazione

```
Flight-Flow/
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
├── report/
│   └── (sorgenti report, img/, main.pdf)
├── scripts/                    #verranno qui aggiunti gli altri due aeroporti
│   ├── schiphol_poller/
│   │   ├── poller.py
│   │   └── requirements.txt
│   ├── common/
│   │   └── flight_schema.py       # schema canonico condiviso dai 3 poller
```

```
│   └── rogue_poller.py            # per la demo di attacco (gitignorato)
├── security/
│   └── generate_certs.sh
└── services/
    ├── API-producer/
    │   ├── Dockerfile
    │   ├── app.py
    │   └── requirements.txt
    ├── dashboard/
    │   ├── Dockerfile
    │   ├── backend.py
    │   ├── requirements.txt
    │   ├── static/
    │   │   ├── script.js
    │   │   └── style.css
    │   └── template/
    │       └── index.html
    ├── notifier/
    │   ├── Dockerfile
    │   ├── notifier.py
    │   └── requirements.txt
    ├── storage/
    │   ├── Dockerfile
    │   ├── storage.py
    │   └── requirements.txt
    ├── stats-aggregator/
    │   ├── Dockerfile
    │   ├── aggregator.py
    │   └── requirements.txt
```

File a livello radice

`.env.example`

Definisce il template delle variabili d'ambiente che il progetto si aspetta di trovare in un file `.env` (che invece non è versionato). Non contiene segreti veri: è un promemoria pubblico dei nomi delle variabili che chi clona il repo deve riempire.

Le variabili minime che deve esporre:

- `KAFKA_SSL_PASSWORD`: pwd condivisa che protegge i keystore/truststore PKCS12. Viene letta sia da `generate_certs.sh` quando crea i PKCS12, sia dal `docker-compose.yaml` per passarla ai broker via environment.
- `SCHIPHOL_APP_ID` e `SCHIPHOL_APP_KEY`: credenziali per l'API di Schiphol, lette dal poller.

Interazioni: file statico non letto da nessun processo. il file .env vero viene invece letto automaticamente da docker compose (che lo inietta nei container

come variabili d'ambiente) e dai poller che girano sull'host (via `python-dotenv` o lettura manuale).

`.gitignore`

Esclude dalla versione tre classi di file: il contenuto di `security/` tranne lo script generatore (i certificati non vanno mai in git), il file `.env` compilato, e i poller di attacco (`scripts/rogue_poller*.py`).

Cartella `security/`

`security/generate_certs.sh`

Definisce la PKI completa del sistema. Uno script shell idempotente (va eseguito una sola volta ma può essere rieseguito per generare tutto da zero) che produce tutti i certificati necessari ad mTLS.

Comportamento atteso:

1. Crea una Certificate Authority interna (`ca.crt` + `ca.key`): coppia RSA con certificato auto-firmato che farà da root of trust per tutto il sistema. Questa CA firmerà tutti gli altri certificati.
2. Per ogni broker Kafka (1,2,3) produce due file PKCS12:
   a. un **keystore** (`kafka.kafka-N.keystore.pkcs12`) che contiene la chiave privata del broker e il suo certificato firmato dalla CA
   - un **truststore** (`kafka.kafka-N.truststore.pkcs12`) che contiene solo il certificato pubblico della CA

   Il workflow per ciascun broker è: genera keypair → genera CSR → firma la CSR con la CA → importa il certificato firmato nel keystore → importa la CA nel truststore.
3. Produce una coppia PEM per i **client Python** (`kafka.client.key` + `kafka.client.certificate.pem`), anch'essa firmata dalla stessa CA. A differenza dei broker, i client Python usano il formato PEM perché è quello che `confluent-kafka-python` accetta nativamente.
4. Organizza i file in sottocartelle: `kafka-1-creds/`, `kafka-2-creds/`, `kafka-3-creds/` (montate nei rispettivi broker) e `client-creds/` (montata in tutti i microservizi Python).

La password dei PKCS12 viene letta dalla variabile d'ambiente `KAFKA_SSL_PASSWORD`. Lo script deve fallire esplicitamente se la variabile non è impostata.

Interazioni: i file generati vengono montati come volumi nel `docker-compose.yml`:

- `./security/kafka-N-creds` → `/etc/kafka/secrets` dentro il broker N
- `./security` intera → `/app/security:ro,z` dentro tutti i microservizi (che accedono ai file via i path sotto `/app/security/client-creds/`)

Il `rogue_poller.py` **non** riceve certificati validi (o prova a usarne uno auto-generato senza passare dalla CA): è lì proprio per dimostrare che non riesce a connettersi.

Cartella `scripts/`

`scripts/common/flight_schema.py`

Definisce lo schema canonico di un evento volo nel sistema. Una dataclass Python `FlightEvent` con tutti i campi normalizzati che il resto del volo si aspetta: identificazione del volo (airport, flight_code, airline), tempi (scheduled, estimated, actual departure + delay_minutes), posizione (gate, terminal, destinazione), stato (SCHEDULED/BOARDING/LAST_CALL/DEPARTED/CANCELLED).

Comportamento che deve esibire:

- fornire la dataclass come contratto unico tra i poller e l'API-producer
- un metodo `to_dict()` per serializzare verso JSON prima del POST HTTP
- idealmente, una validazione di base (es `status` deve essere in un enum definito, `airport` deve essere un codice IATA di 3 caratteri, `scheduled_departure` deve essere un timestamp ISO 8601 in UTC)

Interazioni: importato dal poller Schiphol (e dagli altri due poller degli altri due aeroporti) come output del loro processo di normalizzazione. Strato che nasconde al resto del sistema le differenze tra le API di partenza. Tutto ciò che passa su Kafka è, concettualmente, una serializzazione di FlightEvent.

`scripts/schiphol_poller/poller.py`

Definisce il data source del sistema: un processo long-running che gira sull'host, interroga periodicamente l'API di Schiphol, normalizza i risultati e li invia all'API-producer.

Comportamento atteso

1. Avvio: legge le credenziali Schiphol dal `.env` (`SCHIPHOL_APP_ID`, `SCHIPHOL_APP_KEY`) e l'endpoint dell'API-producer (default http://127.0.0.1:8000/flight)

2. Entra in un loop con polling ogni N secondi (rispettando il rate limit di Schiphol)
3. Ad ogni ciclo interroga l'API di Schiphol filtrando sulle partenze (flightDirection=D) con una finestra temporale che copre le prossime K ore (K da definire).
4. Per ogni volo restituito applica la logica di filtering:
   a. scarta se `is_codeshare` è true
   b. scarta se `serviceType` indica cargo (`C`, `H`) o general aviation (`G`)
   c. tiene solo i voli passeggeri di linea (`J`)
5. I voli superstiti vengono normalizzati in oggetti `FlightEvent` (vengono tradotti i campi proprietari di Schiphol
   nei campi canonici: ad es. `flightName` → `flight_code`, `publicFlightState.flightStates[0]` → `status`, ecc.)
6. Per ogni `FlightEvent` fa una POST HTTP JSON all'endpoint `/flight` dell'API-producer
7. Gestione fallimenti: rate limiting (429 → attesa esponenziale), API-producer down (logga e ritenta al prossimi ciclo, senza crashare), errori di parsing (logga il volo problematico e prosegue).

È importante che il poller sia **idempotente sugli eventi**: se il volo `KL1234` compare in 20 polling consecutivi con lo stesso stato, il producer rimanda 20 messaggi identici. Non è un problema: il sistema deve essere idempotente lato consumer (il dashboard sovrascrive lo stato del volo, lo storage può usare un `upsert` su `flight_code + scheduled_departure`).

Interazioni:

- **Input**: API pubblica di Schiphol (HTTP GET)
- **Output**: API-producer (HTTP POST)
- Non conosce Kafka direttamente — questa è una scelta di design forte, significa che puoi testare il poller anche senza avere il cluster Kafka acceso (basta un echo server).

`scripts/schiphol_poller/requirements.txt`
Solo le dipendenze del poller: `requests` per le chiamate HTTP, `python-dotenv` per leggere `.env`. Non ha `confluent-kafka` perchè il poller non parla mai direttamente con kafka.

`scripts/rogue_poller.py`
Definisce il client malevolo per la demo di sicurezza. Uno script che tenta di pubblivare direttamente sul topic `flight.telemetry` saltando l'API producer, ma senza avere credenziali valide.

Comportamento atteso:

- configura un `confluent_kafka.Producer` puntando direttamente ai broker (porta EXTERNAL 9092/9094/9096 esposta sull'host)
- prova vari scenari di attacco:
  1. connessione in chiaro senza TLS → rifiutata dal broker perché il listener è `SSL`
  2. connessione TLS ma senza certificato client → rifiutata perché `ssl.client.auth=required`
  3. connessione TLS con certificato self-signed non firmato dalla CA del sistema → rifiutata durante l'handshake perché il broker non si fida di quel certificato
- in tutti i casi logga l'errore ricevuto
- tenta di inviare un messaggio malevolo (es. dati falsificati) per dimostrare che *anche con payload ben formato* la connessione non si apre

Interazioni: prova a connettersi ai broker Kafka sulle porte EXTERNAL. Nessun altro componente. È gitignorato (non va nel repo pubblico perché è codice di attacco, seppure didattico — anche se l'ho visto fare in report accademici, meglio non pubblicarlo).

Cartella `services/`

`services/API-producer/app.py`
Definisce il gateway HTTP → Kafka. Un'applicazione Flask che espone endpoint REST per ricevere eventi volo e li ripubblica sul topic Kafka.

Comportamento atteso:

1. Avvio: crea un `confluent_kafka.Producer` configurato con: `bootstrap.servers` (lista completa dei 3 broker), `security.protocol=SSL`, i tre path dei certificati mTLS letti da env (`KAFKA_SSL_CA_LOCATION`, `KAFKA_SSL_CERTIFICATE_LOCATION`, `KAFKA_SSL_KEY_LOCATION`) e soprattutto le impostazioni di durabilità: `acks=all`, `enable.idempotence=True`, `retries` molto alto, `max.in.flight.requests.per.connection=5`, `compression.type=gzip`.

2. Espone l'endpoint `POST /flight` che accetta un JSON conforme a `FlightEvent`. Contratto dell'endpoint:
   a. Valida il JSON contro uno schema Pydantic (una ricopiatura di `FlightEvent`)
   b. Calcola una chiave Kafka: ho scelto `airport:flight_code` per il partitioning: tutti gli update dello stesso volo finiscono nella stessa partizione, quindi vengono ordinati.
   c. Chiama `producer.produce(topic, key=..., value=json)` con ua callback che logga fallimenti/successi
   d. Chiama `producer.poll(0)` per far progredire la coda asincrona
   e. Risponde al client `200 OK` con un body minimale (es: `{"status":"queued"}`)
3. Espone `GET /healtz` che ritorna `200` se il processo è vivo (usato dal healthcheck Docker). Non deve avere troppe logiche, deve essere leggero.
4. Al shutdown chiama `producer.flush()` per garantire che tutti i messaggi in buffer siano spediti prima di uscire.

Interazioni:

- **Input**: riceve chiamate HTTP dai poller (Schiphol per ora, Heathrow/Munich in futuro)
- **Output**: pubblica sul topic `flight.telemetry` del cluster Kafka
- Non ha dipendenze da nessun altro microservizio, è volutamente un *dumb pipe*
- Healthcheck utilizzato da `depends_on` di altri servizi come `dashboard` (aspettano che il producer sia pronto prima di partire)

`services/API-producer/Dockerfile`

Definisce l'immagine del servizio. Base `python:3.11-slim`, installa le deps, copia `app.py`, espone la porta 8000, comando di avvio (`uvicorn app:app --host 0.0.0.0 --port 8000`).

`services/API-producer/requirements.txt`

Dipendenze: `fastapi`, `uvicorn`, `pydantic`, `confluent-kafka`.

`services/storage/storage.py`

Definisce il **consumer di persistenza**: legge eventi da `flight.telemetry` e li salva in MongoDB.

Il comportamento atteso:

1. All'avvio crea un `confluent_kafka.Consumer` con: `group.id=storage-group` (statico — serve per il load balancing quando scali), `enable.auto.commit=False` (commit manuale, fondamentale per at-least-once), `auto.offset.reset=earliest`, e tutta la configurazione mTLS come sopra. Si sottoscrive a `flight.telemetry`.
2. Crea un client `pymongo.MongoClient` verso `mongodb:27017`, seleziona il database `flight_archive` e la collection `departures`.
3. All'avvio, crea (se non esiste) un **indice composito** `{airport: 1, scheduled_departure: 1, flight_code: 1}` unico. Questo indice serve a due scopi: rendere le query efficienti e permettere l'upsert idempotente.
4. Entra nel loop: `consumer.poll(1.0)` → se arriva un messaggio, deserializza il JSON → fa `collection.update_one({filter composito}, {"$set": {...}, "$setOnInsert": {"first_seen": now}}, upsert=True)` → se l'operazione va a buon fine chiama `consumer.commit(message=msg, asynchronous=False)`. L'ordine è cruciale: prima scrivi su Mongo, poi committi l'offset. Se crashi tra le due operazioni, al riavvio rileggerai lo stesso messaggio e l'upsert lo troverà già esistente (idempotenza).
5. Gestisce errori: se Mongo è down, NON committa l'offset e lascia che il messaggio venga riprocessato al prossimo ciclo. Se il JSON è malformato, logga e committa comunque per non bloccare la partizione.

Interazioni:

- **Input**: consuma da `flight.telemetry`
- **Output**: scrive su MongoDB (database `flight_archive`, collection `departures`)
- Nessuna interazione con altri microservizi. Non pubblica mai nulla su Kafka.
- Quando scalato a N repliche, Kafka distribuisce automaticamente le 6 partizioni tra le istanze (questa è la demo di load balancing).

`services/storage/Dockerfile` + `requirements.txt`

Stessa base. Requirements: `confluent-kafka`, `pymongo`.

`services/notifier/notifier.py`

Definisce il **servizio di alerting**: consuma da `flight.telemetry`, applica regole di business, e se rileva condizioni anomale pubblica su `flight.alerts`. È il tuo primo esempio di servizio dual-role (consumer + producer).

Il comportamento atteso:

1. Setup simile a storage ma con `group.id=notifier-group` e in più un `Producer` configurato per pubblicare su `flight.alerts`.
2. Per ogni messaggio consumato applica una serie di **regole** (funzioni che prendono un `FlightEvent` e restituiscono `Alert` o `None`):
   - `high_delay_rule`: se `delay_minutes >= 60` → alert di tipo `SIGNIFICANT_DELAY`
   - `cancellation_rule`: se `status == "CANCELLED"` → alert `FLIGHT_CANCELLED`
3. Per ogni alert generato pubblica su `flight.alerts` con chiave `airport:flight_code`.
4. Commit manuale dell'offset dopo che il producer ha confermato la scrittura (`producer.flush()` periodico).

È importante che la logica delle regole sia **pura e testabile separatamente** — funzioni che prendono un evento e restituiscono un alert senza toccare Kafka. Questo ti permette di fare unit test senza infrastruttura.

Interazioni:

- **Input**: consuma da `flight.telemetry`
- **Output**: pubblica su `flight.alerts`
- Nessun altro consumer legge oggi da `flight.alerts` (il Telegram forwarder è una futura estensione). Ma il topic esiste comunque, viene popolato, e su kafka-ui puoi mostrare i messaggi che scorrono — questo è un punto a favore della tua demo perché mostra che l'architettura è estensibile senza modifiche.
- Scalabile come storage.

`services/notifier/Dockerfile` + `requirements.txt`

Requirements: solo `confluent-kafka`.

`services/stats-aggregator/aggregator.py`

Definisce il **servizio di aggregazione statistica**: consuma `flight.telemetry`, mantiene in memoria contatori rolling per aeroporto, e pubblica periodicamente

un riassunto su `flight.stats`. È il secondo servizio dual-role.

Il comportamento atteso:

1. Setup simile al notifier ma con `group.id=stats-aggregator-group`.
2. Mantiene in memoria una struttura dati indicizzata per aeroporto, del tipo:

   ```
   {
        "AMS": {
          "flights_seen_today": {set di flight_code unici visti},
          "flights_departed_today": {set di flight_code con status DEPARTED},
          "delays": [lista dei delay_minutes dei voli partiti],
          "last_updated": timestamp
        }
      }
   ```

   Il "today" è inteso come giorno UTC corrente; allo scoccare della mezzanotte UTC la struttura va resettata.
3. Per ogni messaggio consumato aggiorna le strutture in memoria. Commit dell'offset dopo l'update.
4. Ha un **thread/task separato** (o un check in loop) che ogni `PUBLISH_INTERVAL_SEC` secondi (default 10) calcola:
   - `total_departed_today` = dimensione del set `flights_departed_today`
   - `avg_delay_minutes` = media dei delay
   - `current_flights_tracked` = dimensione del set `flights_seen_today`
   - (opzionale) chiamata a open-meteo con le coordinate fisse di Schiphol per il meteo corrente

   e pubblica un messaggio su `flight.stats` con chiave = codice aeroporto.
5. Gestisce il reset a mezzanotte UTC azzerando i contatori.

Perché usare memoria invece di Mongo: la memoria è sufficiente perché lo stato è piccolo (qualche centinaia di voli per aeroporto per giorno) e perché in caso di crash il servizio si ricostruisce rileggendo `flight.telemetry` da `auto.offset.reset=earliest` (a patto che la retention del topic sia almeno 24h). Questo è il pattern classico "Kafka as source of truth, services hold derived state".

Interazioni:

- **Input**: consuma da `flight.telemetry`, opzionalmente HTTP call a open-meteo

- **Output**: pubblica su `flight.stats` ogni 10 secondi
- Il dashboard consuma `flight.stats` per mostrare i numeri in testa al riquadro

`services/stats-aggregator/Dockerfile` + `requirements.txt`

Requirements: `confluent-kafka`, `requests` (per open-meteo se lo includi).

`services/dashboard/backend.py`

Definisce il **frontend server**: una web app Flask che consuma da Kafka in background e serve il dashboard HTML + uno stream SSE al browser. È il componente più complesso.

Il comportamento atteso:

1. All'avvio fa partire un **thread background** che crea due consumer Kafka:
   - uno su `flight.telemetry` con `group.id=dashboard-group-{uuid.uuid4()}` (group ID random = ogni dashboard è un gruppo a sé → pattern broadcast)
   - uno su `flight.stats` con lo stesso pattern di group ID random

   L'update è protetto da un `threading.Lock` .Il thread consuma messaggi e aggiorna uno **state in memoria** del tipo:

   ```
   state = {
        "AMS": {
          "flights": {flight_code: FlightEvent dict, ...},  # tutti i voli visti
          "stats": {ultimo messaggio ricevuto da flight.stats}
        }
      }
   ```

2. Espone le route Flask:
   - `GET /` → serve `template/index.html`
   - `GET /static/...` → file statici (Flask lo fa automatico)
   - `GET /healthz` → `200 OK`
   - `GET /stream` → endpoint Server-Sent Events. Ogni 1-2 secondi, prende uno snapshot dello state, lo filtra (per ogni aeroporto: voli con `scheduled_departure > now` ordinati per `scheduled_departure`, primi 40), e lo serializza in JSON `data: {...}\n\n`. Il browser rimane connesso a questo stream e riceve update push.

3. Gestisce la chiusura pulita (signal handler SIGTERM) chiudendo consumer e thread.

Interazioni:

- **Input**: consuma da `flight.telemetry` e `flight.stats` (broadcast)
- **Output verso browser**: HTTP (HTML iniziale) + SSE continuo su `/stream`
- Non pubblica mai su Kafka
- Non tocca MongoDB (il dashboard mostra solo lo stato "vivo" in memoria; per storico serve query direttamente su Mongo — non è ora parte del requisito)
- `depends_on: api-producer` nel compose (per assicurarsi che tutto il chain sia pronto)

`services/dashboard/template/index.html`

Definisce la **struttura HTML** della pagina. Tre riquadri (uno per aeroporto, ma per ora solo quello di Schiphol è attivo), ciascuno con una sezione "stats" in alto e una tabella/lista di voli sotto. Contiene i placeholder (es. `<div id="amsterdam-flights">`, `<div id="amsterdam-stats">`) che JS popolerà.

`services/dashboard/static/script.js`

Definisce il **client-side**. Si connette all'endpoint SSE `/stream` via `EventSource`, parsa ogni messaggio JSON, e aggiorna il DOM. Logica tipica:

- per ogni aeroporto nello snapshot, aggiorna il pannello stats (total departed, avg delay, meteo)
- per ogni aeroporto, renderizza la lista dei 40 voli con codice, compagnia, gate, orario schedulato, ritardo, stato
- evidenzia visivamente gli stati critici (`BOARDING` / `LAST_CALL` in arancione, `DELAYED` in rosso, ecc.)
- gestisce la riconnessione SSE in caso di interruzione

`services/dashboard/static/style.css`

Definisce il **layout** in 3 colonne (grid o flex), colori, tipografia. Niente di logico.

`services/dashboard/Dockerfile` + `requirements.txt`

Requirements: `flask`, `confluent-kafka`.

Ordine di sviluppo consigliato

Ti suggerisco una progressione **bottom-up** con un punto di verifica dopo ogni fase. Idea generale: non scrivere il layer N finché il layer N-1 non funziona ed è testato in isolamento.

Fase 1 — Fondazioni infrastrutturali

Obiettivo: avere un cluster Kafka a 3 broker funzionante e dimostrabile sicuro, senza ancora nessun servizio applicativo.

1. Scrivi `generate_certs.sh` e lo esegui. Verifichi che produca tutti i file attesi nelle cartelle `kafka-N-creds/` e `client-creds/`.
2. Scrivi `docker-compose.yml` includendo **solo** i 3 broker Kafka + kafka-ui + mongodb. Lancia `docker compose up`.
3. Verifica dal log che i broker si scambino il quorum KRaft correttamente, e che kafka-ui sulla 8080 mostri 3 broker vivi.
4. Da linea di comando (usando il `kafka-topics` dentro un container broker), crea manualmente i topic `flight.telemetry` (6 partizioni, RF=3), `flight.alerts` (3 partizioni, RF=3), `flight.stats` (3 partizioni, RF=3). Verifica su kafka-ui.
5. **Test di fault tolerance (anticipato)**: `docker compose stop kafka-2`, verifichi su kafka-ui che le partizioni che avevano kafka-2 come leader siano state riassegnate ai superstiti.

A questo punto hai un cluster Kafka production-grade. È già metà del progetto.

Fase 2 — Ingestion

Obiettivo: essere in grado di pubblicare eventi sul cluster da un client Python con mTLS, e verificare che arrivino.

1. Scrivi `scripts/common/flight_schema.py`.
2. Scrivi `services/API-producer/` (Dockerfile + app.py + requirements). Aggiungi il servizio al compose.
3. Testa l'endpoint con `curl`: invia un `FlightEvent` di esempio a `http://localhost:8000/flight`. Verifica su kafka-ui che il messaggio compaia nel topic `flight.telemetry`.
4. Verifica anche il caso patologico: spegni un broker e manda un POST, deve continuare a funzionare.

Ora hai l'ingestion funzionante indipendentemente dal poller.

Fase 3 — Primo consumer (il più semplice)

Obiettivo: chiudere il loop completo produttore → broker → consumer → sink.

1. Scrivi `services/storage/`. Aggiungi al compose insieme a `mongodb` (se non c'era già).
2. Manda eventi con curl come prima, verifica che compaiano in MongoDB (`mongosh` dentro il container: `docker compose exec mongodb mongosh flight_archive --eval "db.departures.find()"`).
3. Test di idempotenza: manda lo stesso evento due volte, verifica che in Mongo ci sia un solo documento (grazie all'upsert + indice unico).
4. **Test di resilienza consumer**: durante l'invio, fai `docker compose restart storage`. Verifica che al ritorno riprenda dall'ultimo offset senza perdere nulla.

Fase 4 — Il poller vero

Obiettivo: sostituire il curl manuale con una sorgente dati reale.

1. Registrati sul portale developer di Schiphol, ottieni APP_ID e APP_KEY.
2. Scrivi `scripts/schiphol_poller/poller.py`. Testa prima la parte di fetch+normalize offline (stampando i `FlightEvent` generati su stdout, senza chiamare l'API-producer). Qui verifichi che il filtering di codeshare/cargo funzioni come vuoi.
3. Poi colleghi la POST all'API-producer. Lasci girare 10 minuti e verifichi che: i voli arrivino su `flight.telemetry`, siano persistiti su Mongo, siano ben formati.

Fase 5 — Servizi dual-role

Obiettivo: aggiungere i servizi più interessanti dal punto di vista architetturale.

1. Scrivi `services/notifier/`. Verifica che alert compaiano su `flight.alerts` quando ricevi voli con ritardi alti o cancellati.
2. Scrivi `services/stats-aggregator/`. Verifica su kafka-ui che `flight.stats` si popoli ogni 10 secondi.

Fase 6 — Dashboard

Obiettivo: l'interfaccia utente che chiude il sistema.

1. Scrivi il `backend.py` del dashboard con l'endpoint `/stream`, ma **prima** testalo con un client fittizio (`curl http://localhost:8500/stream` ti deve mostrare lo stream di JSON).
2. Scrivi HTML+CSS statici senza logica JS. Verifichi che la pagina si carichi ma sia vuota.
3. Aggiungi `script.js` che consuma SSE e popola i DOM. Verifichi iterativamente.

Fase 7 — Demo dei requisiti non funzionali

Obiettivo: preparare gli scenari per la presentazione.

1. Scrivi `scripts/rogue_poller.py` con i tre scenari di attacco. Verifica che tutti e tre vengano rifiutati.
2. Prepara gli script/comandi per le tre demo:
   - fault tolerance: `docker compose stop kafka-2` con il sistema in esecuzione
   - load balancing: `docker compose up -d --scale storage=3`, guarda kafka-ui
   - security: `python scripts/rogue_poller.py`, mostra i log dell'errore

Fase 8 — Estensioni future

L'aggiunta di Heathrow/Munich sarà semplicemente: creare `scripts/heathrow_poller/` e `scripts/munich_poller/` con lo stesso contratto di output (dict conformi a `FlightEvent`). Nessuna modifica al resto del sistema. Idem per il Telegram forwarder: aggiungere una cartella `services/telegram-forwarder/` e un blocco nel compose che consuma `flight.alerts`.